实验二　Shell 程序设计实验报告

实验环境　VMware 虚拟机 · Ubuntu · Bash · Xshell
实验目录　`~/linux-lab2`

一、实验目的

通过父子 Shell 观察变量继承，通过引号和反引号比较不同的展开行为。编写五个脚本，练习 `case`、`if`、`for`、`while` 和函数调用，并用实际输入验证各分支及边界情况。

二、变量与引号

1. Shell 变量与环境变量

在父 Shell 中给 `var1` 和 `var2` 赋值，只执行 `export var2`，再启动子 Bash 检查两个变量。

```
unset var1 var2
var1="welcome to use local variable"
var2="I am global variable"
export var2
bash --noprofile --norc -c 'printf "child var1=[%s]\nchild var2=[%s]\n" "$var1" "$var2"'
printf 'parent var1=[%s]\nparent var2=[%s]\n' "$var1" "$var2"
```

预期子 Bash 的 `var1` 为空，`var2` 为 `I am global variable`。父 Shell 中两者都有值。本项保留复现命令和预期结果，未附运行截图。

2. 引号与命令替换

截图中，`echo *` 和 `echo $var3` 都展开为 `alpha.txt beta.txt`。`echo "*"` 与 `echo "$var3"` 输出星号，`echo '$var3'` 输出变量名本身。用反引号包住命令时，Bash 执行命令并用其输出替换原位置的表达式。

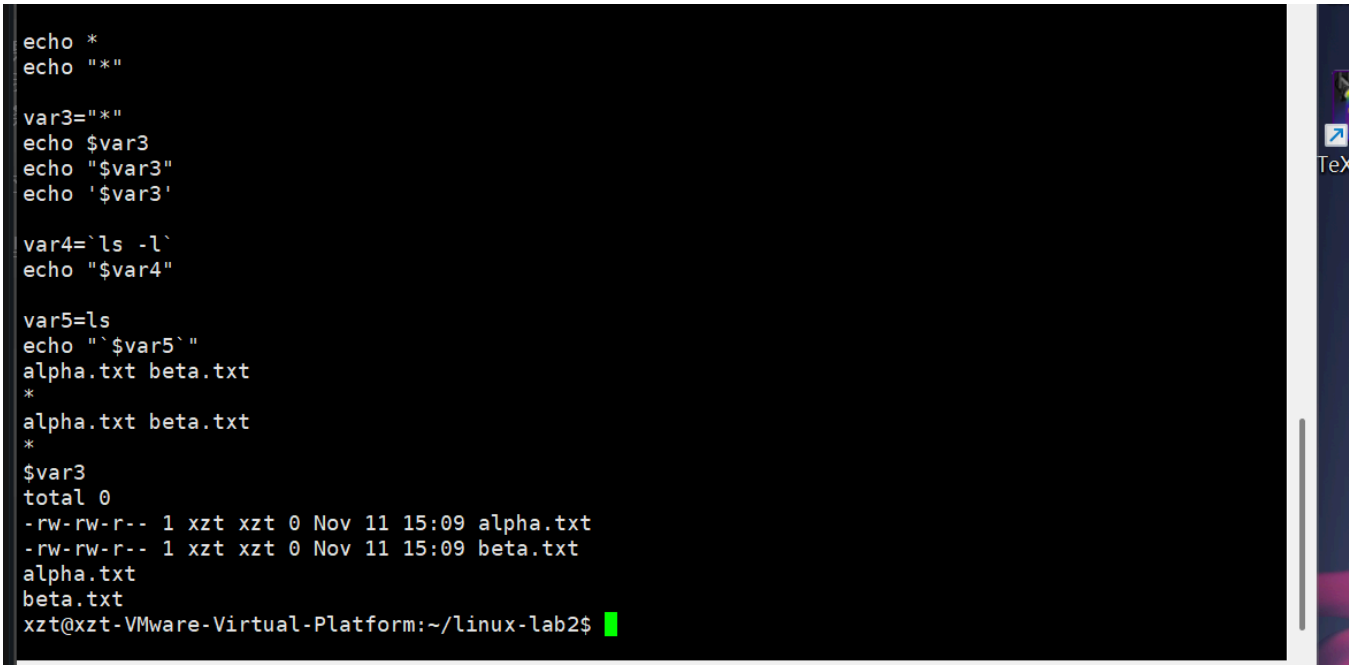


图 1　引号与命令替换的终端输出

三、脚本实现与运行结果

1. 根据系统时间输出问候语（greet.sh）

使用 `date +%H` 取得 Ubuntu 的当前小时，再由 `case` 匹配时段。按指导书，`01`～`11` 为上午，`12`～`17` 为下午，`00` 和 `18`～`23` 为晚上。脚本也接受一个两位小时参数，便于逐一验证。

截图中系统小时为 `15`，直接运行输出 `Good afternoon!`。分别传入 `09`、`15`、`20`，得到上午、下午和晚上的问候。

```
-rw-rw-r-- 1 xzt xzt 0 Nov 11 15:09 beta.txt
alpha.txt
beta.txt
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ cat > greet.sh <<'EOF'
#!/bin/bash
hour=${1:-$(date +%H)}

case "$hour" in
    0[1-9]|1[01]) echo "Good morning!" ;;
    1[2-7]) echo "Good afternoon!" ;;
    00|1[89]|2[0-3]) echo "Good evening!" ;;
    *) echo "Use an hour from 00 to 23." >&2; exit 1 ;;
esac
EOF
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ date +%H
15
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash greet.sh
Good afternoon!
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash greet.sh 09
Good morning!
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash greet.sh 15
Good afternoon!
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash greet.sh 20
Good evening!
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$
```

图 2　系统时间与三个问候分支的运行结果

2. 比较两个整数（compare.sh）

用 `read -r` 读取两个整数，用 `-gt` 和 `-lt` 做数值比较，最后由 `else` 处理相等。分别输入 `8`、`3`、`-2`、`5` 和 `7`、`7`，三个分支均输出预期结果。

```
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash compare.sh
Enter the first integer: 8
Enter the second integer: 3
8 is greater than 3
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash compare.sh
Enter the first integer: -2
Enter the second integer: 5
-2 is less than 5
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ bash compare.sh
Enter the first integer: 7
Enter the second integer: 7
7 is equal to 7
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$
```

图 3　大于、小于和等于三个分支的运行结果

3. 从若干数中寻找最小值（minimum.sh）

将数组的首项 `8` 作为当前最小值，再用 `for` 遍历 `8 2 18 0 -3 87`。遇到更小的元素时更新，最小值依次变为 `2`、`0`、`-3`。运行输出为 `-3`。

```
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ cat > minimum.sh <<'EOF'
#!/bin/bash
numbers=(8 2 18 0 -3 87)
smallest=${numbers[0]}

for number in "${numbers[@]}"; do
    if (( number < smallest )); then
        smallest=$number
    fi
done

echo "$smallest"
EOF

bash minimum.sh
-3
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$
```

图 4　遍历数组求最小值

4. 统计当前目录下的可执行文件（count_exec.sh）

用 `for file in *` 遍历当前目录的非隐藏条目。只有同时满足 `-f`（普通文件）和 `-x`（当前用户可执行）的条目才计入。在测试目录中，`run.sh` 满足两个条件，文本文件没有执行权限，`subdir` 是目录。脚本输出 `Total of 1 files executable`。

```
#!/bin/bash
count=0

for file in *; do
    if [[ -f "$file" && -x "$file" ]]; then
        count=$((count + 1))
    fi
done

echo "Total of $count files executable"
EOF
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$ mkdir -p count-test/subdir
printf '#!/bin/bash\nexit 0\n' > count-test/run.sh
printf 'test text\n' > 'count-test/notes file.txt'
chmod +x count-test/run.sh
chmod a-x 'count-test/notes file.txt'

ls -l count-test
(cd count-test && bash ../count_exec.sh)
total 12
-rw-rw-r-- 1 xzt xzt   10 Nov 11 15:14 'notes file.txt'
-rwxrwxr-x 1 xzt xzt   19 Nov 11 15:14  run.sh
drwxrwxr-x 2 xzt xzt 4096 Nov 11 15:14  subdir
Total of 1 files executable
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$
```

图 5　测试目录权限列表与可执行文件统计

5. 用函数判断素数（prime.sh）

函数 `is_prime` 先排除小于 `2` 的整数，再用 `while` 从 `2` 开始试除。只需检查到平方根，因为合数至少有一个因子不超过其平方根。函数以 `return 0` 表示判断成立，调用方用 `if is_prime "$1"` 选择输出。

测试结果显示，`17` 和 `2` 是素数，`18` 和 `1` 不是素数。其中 `1` 和 `2` 用来验证素数判定的边界。

```
    return 0
}

if [[ $# -ne 1 || ! $1 =~ ^-?(0|[1-9][0-9]*)$ ]]; then
    echo "Usage: bash prime.sh INTEGER" >&2
    exit 1
fi

if is_prime "$1"; then
    echo "$1 is a prime!"
else
    echo "$1 is not a prime!"
fi
EOF

bash prime.sh 17
bash prime.sh 18
bash prime.sh 1
bash prime.sh 2
17 is a prime!
18 is not a prime!
1 is not a prime!
2 is a prime!
xzt@xzt-VMware-Virtual-Platform:~/linux-lab2$
```

图 6　素数、合数与边界值的运行结果

四、问题分析与实验心得

- **从指导书示例录入脚本。** Bash 变量赋值的等号两侧需要紧贴变量名和值，注释使用 `#`。比较脚本将分支输出中的变量名统一为 `first`，以匹配读取输入时使用的名称。
- **区分目录权限与文件执行权限。** `ls -l` 显示子目录也带有 `x`，该权限表示可进入目录。统计时同时检查文件类型和执行权限，得到预期的计数 `1`。
- **用不同输入验证分支。** 指定小时可以在同一次实验中覆盖三个问候分支。比较脚本使用大于、小于、相等三组输入，素数脚本加入 `1` 和 `2`，分别检查常规情况和边界。

这次实验让我学会从输出判断变量何时被展开，也学会了用针对性的输入检查脚本是否走到了预期分支。